Programación 1 — Unidad 10: Recursividad

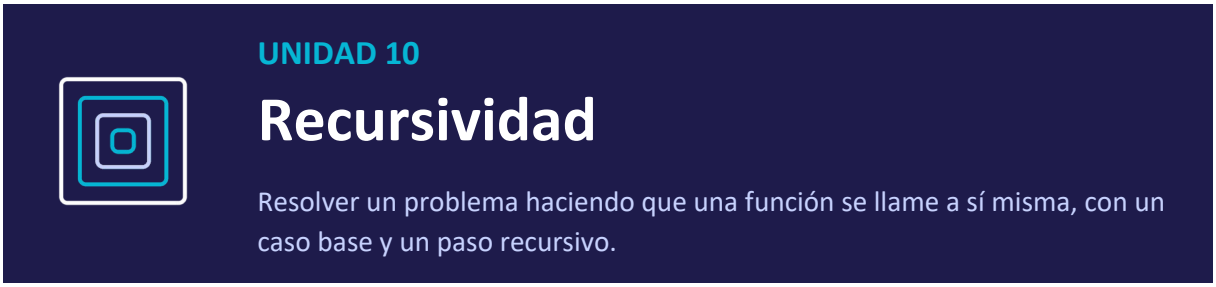
Actividad 1: Introducción al concepto de recursividad

Índice

10. Recursividad

– 10.1. Los dos componentes de una función recursiva

– 10.2. Ejemplo clásico: el factorial



10. Recursividad

La **recursividad** es una técnica en la que una función **se llama a sí misma** para resolver un problema. Suena raro al principio —¿una función que se usa a sí misma?—, pero es una forma muy elegante de resolver ciertos problemas. La idea es que cada llamada resuelve una parte más pequeña, y el conjunto de todas las llamadas llega a la solución final. Es otra manera de lograr repetición, sin usar bucles.

10.1. Los dos componentes de una función recursiva

Toda función recursiva bien hecha tiene dos partes imprescindibles. El **caso base** (o condición de corte) es la situación más simple, que se resuelve directamente y **detiene** la recursión. El **paso recursivo** es donde la función se llama a sí misma con una versión más pequeña del problema, acercándose al caso base.

```
def cuenta_regresiva(n):
    if n == 0:
        print("¡Despegue!")
    else:
        print(n)
        cuenta_regresiva(n - 1)

cuenta_regresiva(3)
```

Qué hace el código:

- `if n == 0:` es el **caso base**: cuando `n` llega a 0, muestra «¡Despegue!» y no se vuelve a llamar. Eso frena todo.
- El `else` es el **paso recursivo**: muestra el número actual y se llama a sí misma con `n - 1` (un problema más chico).
- `cuenta_regresiva(3)` muestra 3 y se llama con 2; esa muestra 2 y se llama con 1; y así hasta el caso base.
- El resultado en pantalla es: 3, 2, 1, ¡Despegue! Cada llamada es como una muñeca rusa más chica.

Recursión: pila de llamadas de cuenta_regresiva(3)



al llegar al caso base, las llamadas se resuelven en orden inverso

Cada llamada genera una nueva, más pequeña, hasta llegar al caso base; recién entonces se resuelven en orden inverso.

Analogía

La recursividad es como las muñecas rusas (matrioskas): para llegar al centro, abrís una muñeca y adentro hay otra más chica, y otra, y otra… hasta la más pequeña que ya no se abre (el caso base). Recién ahí empezás a cerrarlas todas, en orden inverso.

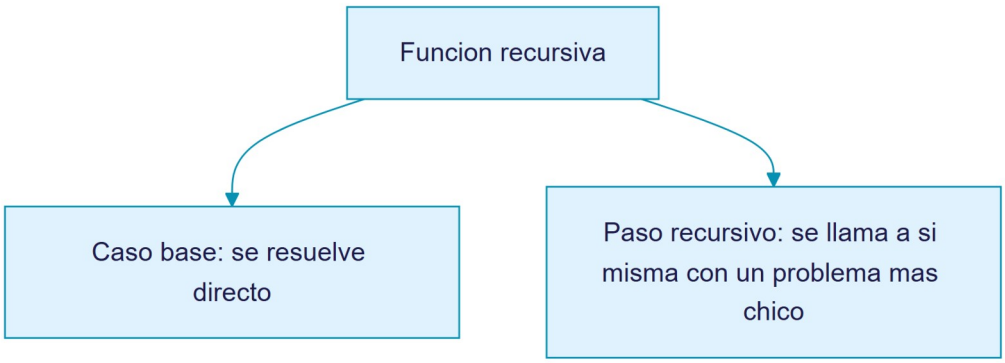


Diagrama 10.1 generado con Mermaid.

10.2. Ejemplo clásico: el factorial

El factorial de un número *n* (escrito *n!*) es el producto de todos los enteros desde 1 hasta *n*. Se define de forma naturalmente recursiva: *n! = n × (n−1)!*, y *0! = 1*. Es el ejemplo más típico para entender la recursión.

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)

print(factorial(5))   # 120
```

Qué hace el código:

- `if n == 0: return 1` es el caso base: el factorial de 0 es 1, y ahí se detiene.
- `return n * factorial(n - 1)` es el paso recursivo: multiplica `n` por el factorial del número anterior.
- Así, `factorial(5)` se transforma en `5 * factorial(4)`, que a su vez es `5 * (4 * factorial(3))`, y así sucesivamente.
- Cuando llega a `factorial(0)` (que vale 1), toda la cadena se resuelve hacia atrás y da 120.

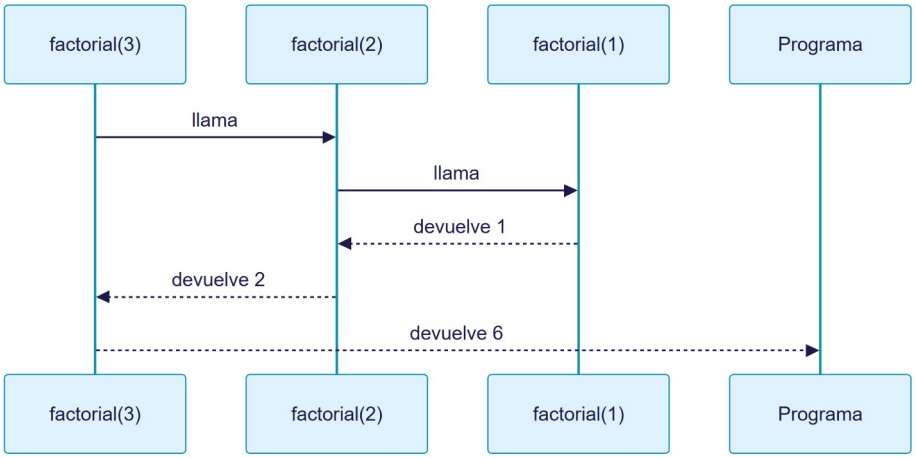


Diagrama 10.2 generado con Mermaid.